DA6400 Reinforcement Learning - Programming Assignment 2

Team Information

Name	Roll Number	Contribution
G C V Sairam	DA25M012	Parts 1, 2, 4a, 4b, 4d, Bonus
G Krishna Chaitanya	DA25M011	Parts 1, 3, 4a, 4c, 4d, Bonus
Nikhil Enugu	DA25M010	Parts 3, 4a, 4b, 4c, 4d

Code Repository

The full implementation and instructions to run the code are available at:

`https://drive.google.com/drive/folders/1ywyqrIM_Ql3o5Spo5uwoPCIXm3OG51bl`

The repository contains the python files for the implementation of the code, the episode returns for each experiment stored as a .npy file with appropriate naming convention, a README file for the setup and execution instructions and a requirements.txt file for python dependencies.

Deep-Q Networks on Mountain Car

Part 1 : Python Implementation

For the Q-network, we used a two-layer MLP with hidden sizes 64 and 32, both followed by ReLU activations, taking the 2-dimensional observation (position, velocity) as input and the output layer gives the Q-values for each of the 3 discrete actions. We use Kaiming(He) initialization for weights in all layers.
We use an ε-greedy policy with decaying ε.

Hyperparameter	Value
Optimizer	Adam with learning rate = 10^{-3}
Replay buffer size	20,000
Batch size	32
Target network update	Hard Updates every 50 steps
ε initial	0.95
ε decay	Linear decay with minimum at 0.05
Episodes per seed	60
ρ	1

Table 1: DQN hyperparameters

The ε decay schedule decreases ε linearly each episode by $\Delta = \frac{(\varepsilon_{\text{start}} - 0.05) \times 1.25}{N_{\text{episodes}}}$, floored at 0.05. The factor of 1.25 ensures ε reaches its minimum value at around the 48th episode, so the last 12 episodes will have the same epsilon value 0.05.

Instead of updating the network at every environment step, we perform a training update once every 4 timesteps. This reduces computational overhead while still maintaining sufficient learning progress. This ensures a balance between data collection and learning.

Part 2: Vanilla DQN Implementation

Part 2a : Results

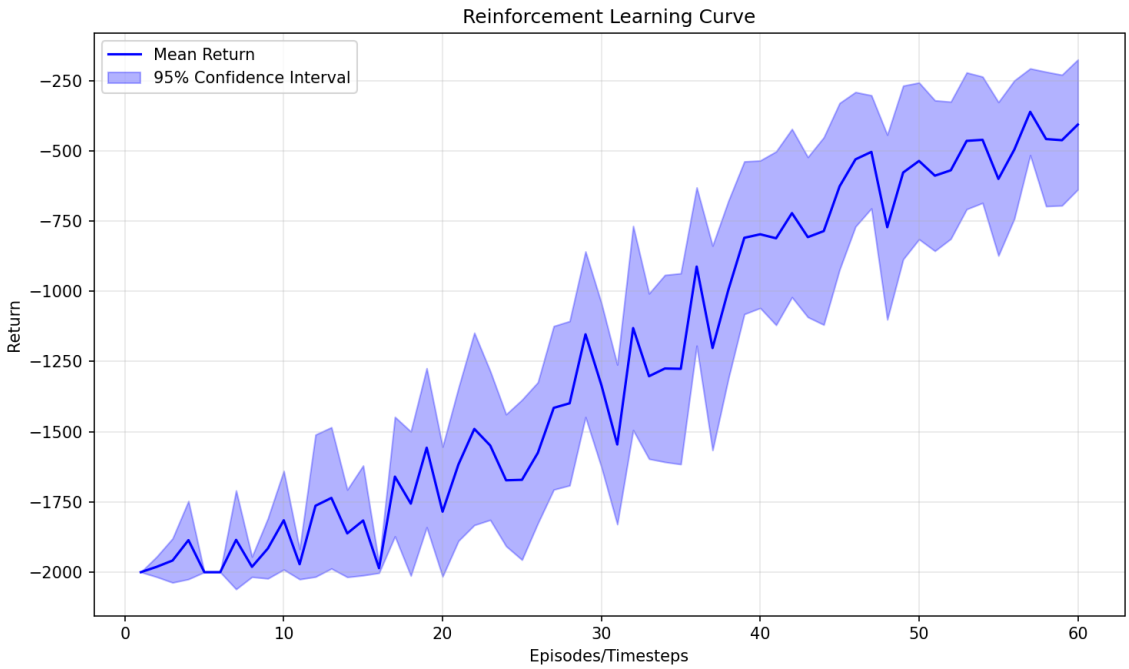


Figure 1: SARSA vs Q-Learning Online Performance

Figure 1 shows the learning curve for the DQN network with truncation length 2000. The agent starts near -2000 (always hitting truncation) and steadily improves, reaching a mean return of around -400 to -500 by episode 60.

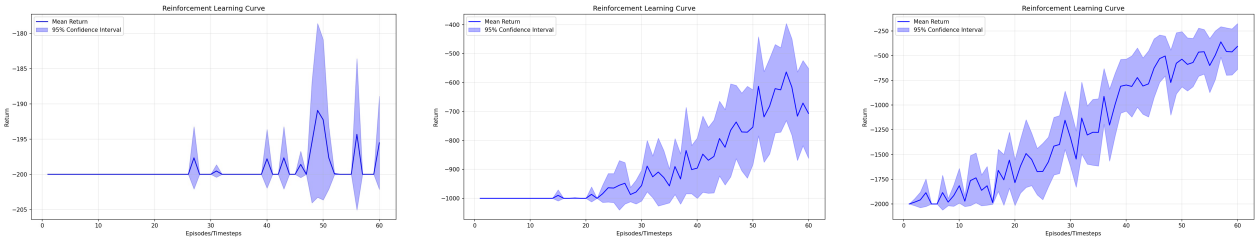
The learning curve shows a clear upward trend but with a wide confidence interval. This is because the default reward structure of MountainCar is very sparse, so the agent gets no signal until it reaches the goal for the first time. Some seeds discover the goal early and converge quickly, while others never reach it within 60 episodes and stay stuck at -2000 throughout. This split pulls the mean down and inflates the variance, making overall performance look worse than it actually is for the converging seeds.

Part 2b : Optimal Behaviour

The car cannot reach the goal by directly accelerating toward it since the acceleration is too slow to climb the hill in one go. The optimal policy instead learns to do back-and-forth oscillation to gain momentum slowly and then swings rightward when it reaches enough velocity to reach the goal.

Part-3 : Experimentation on Vanilla DQN with various truncation lengths

Part 3a : Plots for various truncation lengths



(a) Truncation = 200 (b) Truncation = 1000 (c) Truncation = 2000

Figure 2: DQN learning curves under different truncation lengths

Part 3b : Comparison

Figure 2 compares DQN under three truncation lengths: 200, 1000, and 2000.

Truncation = 200 (default): The agent is almost entirely stuck. The mean return hovers at -200 throughout, with occasional spikes in the middle. Since the episode ends after just 200 steps, the agent rarely experiences a successful trajectory, so the replay buffer does not have any useful signals.

Truncation = 1000: Learning begins after a few episodes and improves steadily, reaching roughly -700 by episode 60. The agent has enough time per episode to occasionally stumble upon the goal, generating the positive learning signal needed to start the learning.

Truncation = 2000: This gives the best and most consistent improvement, starting from -2000 and climbing to around -400 mean by episode 60. Having more timesteps gives the agent significantly more chances per episode to discover the goal, leading to faster and more stable learning overall.

Part 3c : Reason for Truncation

The original MountainCar formulation had no episode limit, so an agent could just roam indefinitely without ever reaching the goal, thus getting stuck. Gymnasium introduced truncation primarily for practical reasons: it prevents experiments from running for infinite time when a policy fails to solve the task. This allows to have an upper limit on the time taken for training the algorithm based on number of timesteps and number of episodes and seeds.

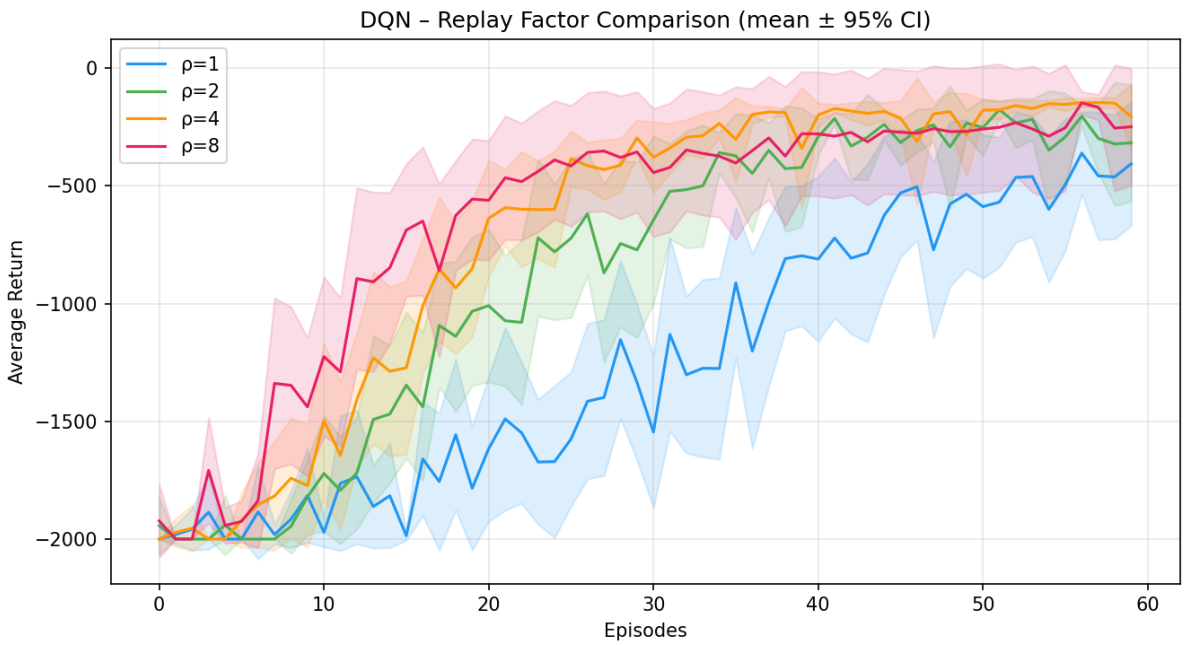
Part 3d : Disadvantages of Truncation

Truncation is harmful when it is too short, as the agent may never encounter a reward signal during the early exploration phase. If the episode ends before the agent accidentally reaches the goal even once, the Q-network has nothing useful to learn from and no useful training happens and this is exactly what we see when the truncation length was set to 200.

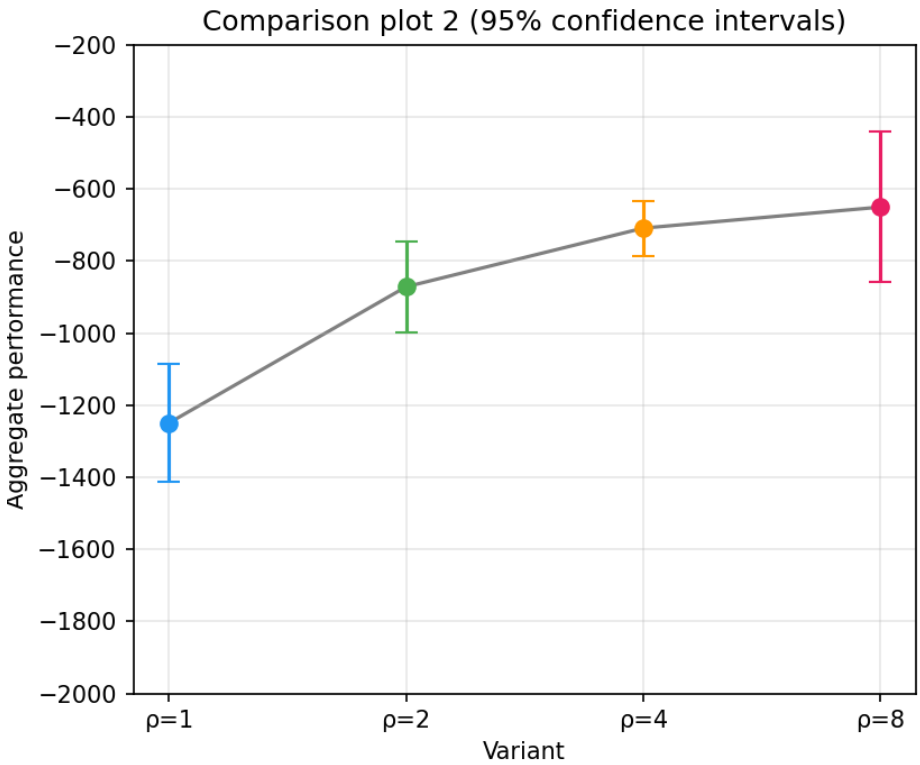
An appropriate truncation length should be long enough that the agent is likely to stumble upon the goal at least once in the first few episodes through random exploration. For this case, this means the truncation length should be atleast 1000 or 2000 steps, giving a randomly acting agent a reasonable chance of reaching the goal by chance and generating the first reward signal needed to start learning.

Part-4 : Experimentation on Vanilla DQN with various Replay Factor

Part 4a : Plots for various Replay Factors



(a) Learning curves



(b) Aggregate performance

Figure 3: Comparison of ρ values

(i) Comparison of ρ values

Figures 3a and 3b show the learning curves and aggregate performance for $\rho \in \{1, 2, 4, 8\}$. All variants start near -2000 and improve over 60 episodes, but higher ρ values learn significantly faster and reach better final performance. $\rho = 1$ is the slowest, barely reaching -500 by episode 60, while $\rho = 4$ and $\rho = 8$ converge much earlier, reaching to an average reward value of -200 to -300.

(ii) Statistical significance

From the 2nd plot, we can see that the differences are statistically significant between $\rho = 1$ and the higher values. The confidence intervals in the aggregate plot do not overlap between $\rho = 1$ and $\rho = 4$ and $\rho = 8$. However, $\rho = 4$ and $\rho = 8$ have overlapping Confidence Intervals, suggesting the gains from increasing ρ from 4 to 8 are not statistically significant.

(iii) Reason for the differences

A higher replay factor means the Q-network is updated ρ times per every call of **train_dqn()** function, getting more updates from each meaningful transition. This results in faster convergence, as seen in the learning curves where $\rho = 4$ and $\rho = 8$ reach near-optimal performance faster than $\rho = 1$. The final performance also improves slightly with ρ given a fixed number of episodes, but going from $\rho = 4$ to $\rho = 8$ provides only a marginal improvement over $\rho = 4$. This is because excessive reuse of the same transitions can lead to overfitting on stale data in the replay buffer, limiting further gains.

(iv) When is increasing ρ advantageous/disadvantageous?

In the case where collecting real-world data is expensive and slow or where the data itself is scarce, a higher ρ value is very beneficial(Example: Robotics).
However, if the environment is a fast simulator and data collection is cheap, a high ρ wastes compute on repeated updates when getting new samples would be more informative.(Example: Online Trading)
A high ρ value can also destabilize early training if the replay buffer does not yet contain enough diverse experiences, causing the network to overfit to a narrow distribution of experiences.

(v) Relation to planning

DQN with $\rho > 1$ loosely relates to the definition of planning in that the agent performs multiple updates using previously collected transitions, which is like reusing past experience to improve the performance. And each additional update per timestep is analogous to simulating extra experience from this model.
But unlike true planning methods, this does not use an explicit model of the environment to generate new trajectories. So, DQN with higher ρ has some benefits of planning methods but not the same since it is still model-free.

Part 4b: Distribution of Performance

(i) Shape of distributions

Figure 4 shows the distributions of aggregate performance across seeds. The $\rho = 1$ distribution looks bimodal — one mode near -2000 (unconverged seeds) and another around -1250 (partially converged seeds). As ρ increases, the distributions shift rightward and become more unimodal: $\rho = 2$ shows a dominant peak near -800, while $\rho = 4$ is the most concentrated with a sharp peak around -750. $\rho = 8$ has a wider, flatter distribution skewed toward higher returns, but with a long left tail suggesting some seeds still underperform.

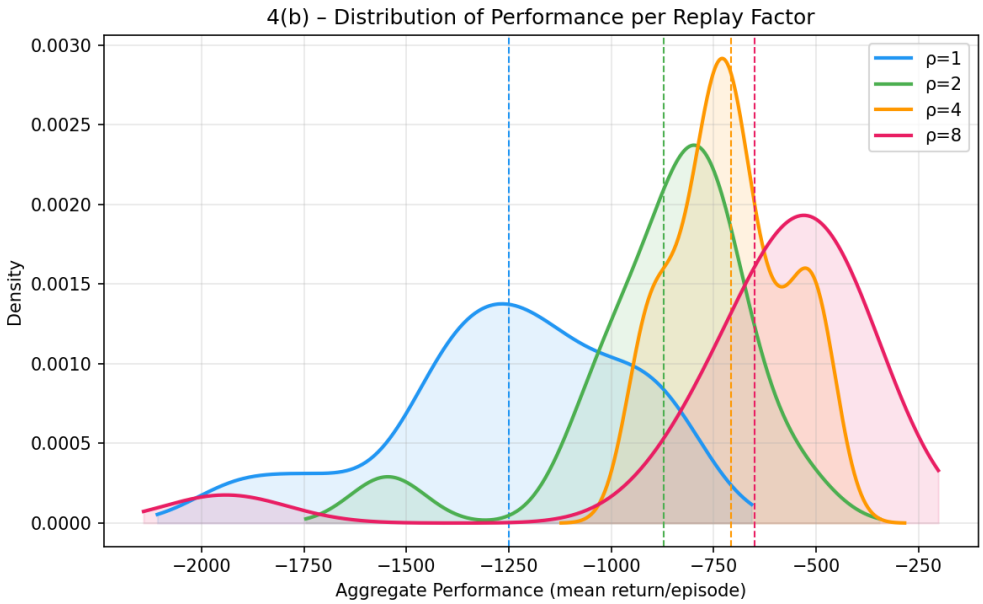


Figure 4: Performance Distributions

(ii) Inferences

For $\rho = 1$, the distribution is wide indicating a high number of runs which performed very badly. As ρ increases, the distribution shifts to the right, which indicates improvement in performance and reduced variability.
And the distributions are not unimodal, this indicates that runs/seeds are unstable and divergent.

(iii) When are distribution plots more useful than learning curves?

Distribution plots are especially useful when algorithm performance is highly variable across runs. For example, in sparse reward environments, multi-goal tasks, or when comparing algorithms with very different exploration strategies. A learning curve averages over seeds and can mask whether an algorithm is reliably good or just occasionally lucky. In these cases, looking at distribution plots is better.

Part 4c: Tolerance Intervals

(i) Tolerance Interval Curve

Tolerance intervals are statistical intervals that capture the range within which a specified proportion of the population is expected to lie, with a given confidence level. Here, they are used to measure the spread of performance across different runs, which tells us how consistent or variable the agent's performance is.

(ii) Comparison of Tolerance Intervals

Figure 5 shows the mean return with $(\alpha = 0.05, \beta = 0.9)$ tolerance intervals. For $\rho = 1$, the intervals are much wider, especially in early training, indicating high variability and less consistent learning across seeds. Higher ρ values exhibit narrower intervals as training progresses, suggesting more stable and reliable performance. Compared to part (a), which only showed average returns, the tolerance intervals reveal the spread and worst-case behavior of the agent, highlighting that higher ρ not only improves mean performance but also reduces variability and leads to more consistent learning outcomes.

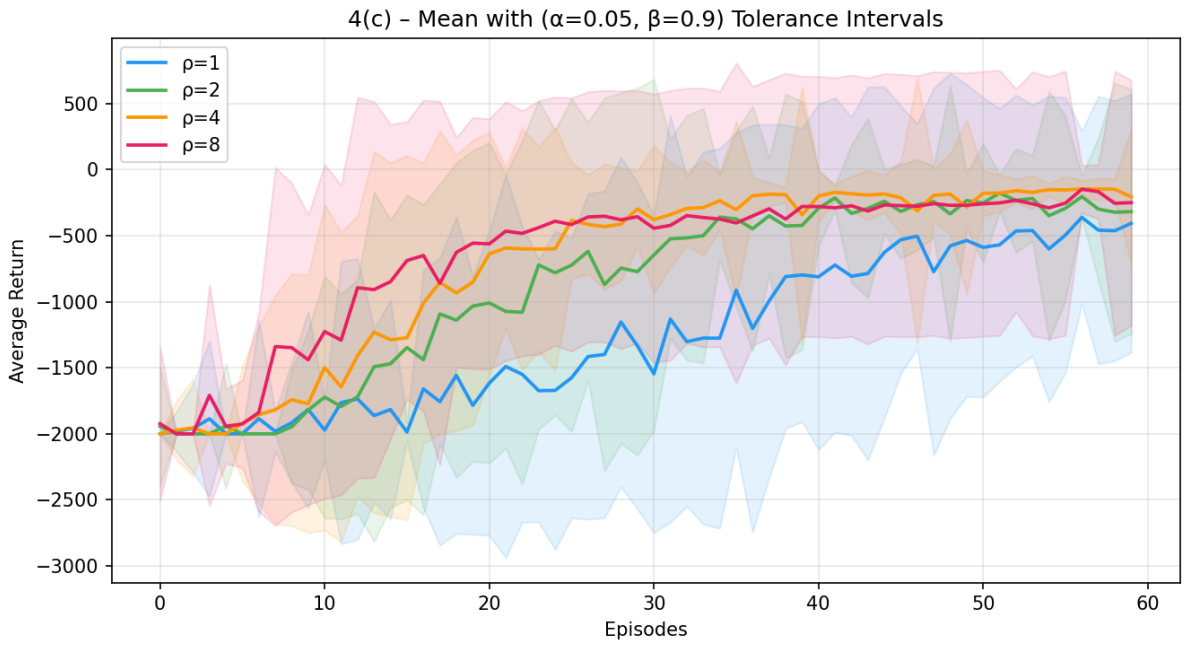


Figure 5: Tolerance Intervals

(iii) Reliability and worst-case performance

$\rho = 1$ has the widest tolerance band, with the lower bound dropping below -2500 in some episodes. Even though getting a reward of less than -2000 is not possible, the tolerance interval goes till -2500 since it assumes a normal distribution for the values. In contrast, $\rho = 4$ has a noticeably tighter band, indicating more reliable and consistent performance across seeds. $\rho = 8$ has a wide band despite a good mean, suggesting it is less robust. The worst-case performance (lower tolerance bound) clearly improves as ρ increases from 1 to 4, but $\rho = 8$ shows no further improvement in robustness.

(iv) Fundamental difference between tolerance and confidence intervals

A confidence interval measures uncertainty around the mean. A tolerance interval measures the spread of individual observations. Confidence Intervals are useful for comparing while testing, and tolerance intervals are useful for understanding deployment reliability.

Part 4d : Sensitivity Analysis

(i) Mini-batch size sensitivity

Both $\rho = 1$ and $\rho = 4$ are sensitive to batch size. Smaller batch sizes lead to noisy updates and poorer performance, while larger batch sizes improve stability up to a point. DQN with $\rho = 4$ is more sensitive, as multiple updates amplify gradient noise when the batch size is small. Overall, a moderate batch size provides the best trade-off between stability and learning efficiency.

(ii) Target network refresh rate sensitivity

Performance depends strongly on the target update frequency. Very frequent updates lead to instability due to rapidly changing targets, while infrequent updates slow learning. DQN with $\rho = 4$ is relatively more stable across a wider range of refresh rates, but both variants perform best at intermediate values.

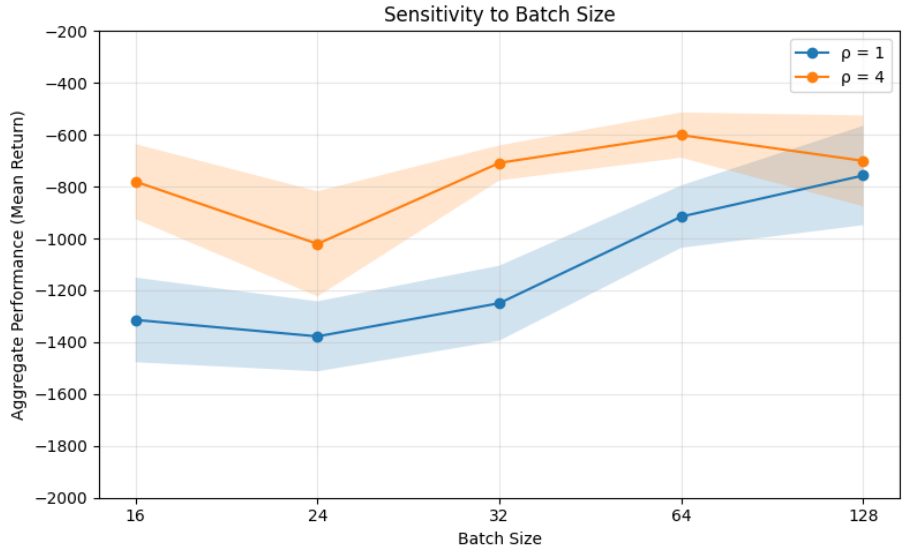


Figure 6: Sensitivity plot for Batch Size

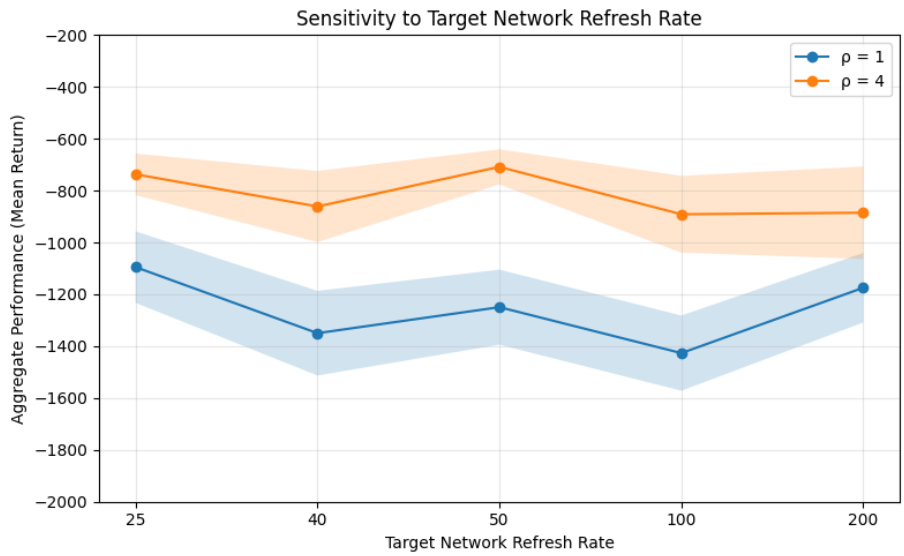


Figure 7: Sensitivity Plot for Target Network Refresh Rate

(iii) Insights for deployment

Higher replay factor improves sample efficiency but increases sensitivity to hyperparameters. While $\rho = 4$ can achieve better performance, it requires careful tuning of batch size and target update frequency. Thus, robust deployment favors moderate hyperparameter choices to balance stability and efficiency.

Bonus : Effect of PER on replay factor (ρ)

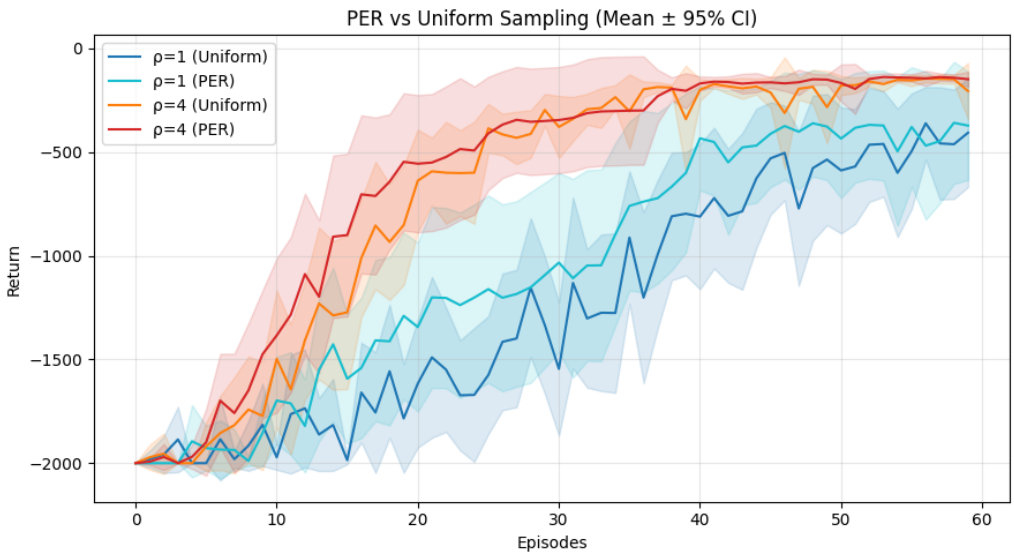


Figure 8: Learning Curves with and without PER for rho = 1 and 4

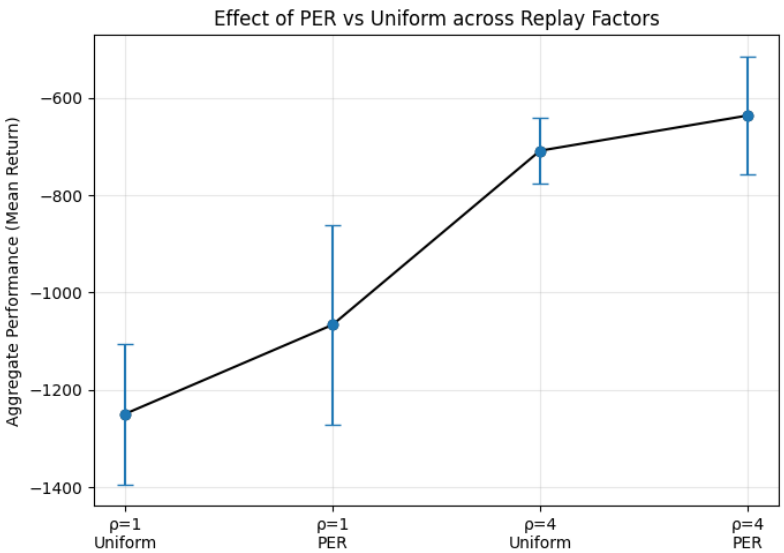


Figure 9: Aggregate Performance with and without PER

We compare uniform sampling and Prioritized Experience Replay (PER) for $\rho \in \{1, 4\}$ using mean learning curves and aggregate performance.

From the learning curves, PER consistently improves sample efficiency, especially for $\rho = 1$, where it leads to faster learning and better early performance. For $\rho = 4$, both uniform and PER perform well, but PER still provides a noticeable improvement in convergence speed and stability.

From the aggregate performance plot, increasing ρ improves performance under both sampling strategies. However, the relative improvement from $\rho = 1$ to $\rho = 4$ is smaller when using PER compared to uniform sampling. This indicates that with PER, there is still a benefit of increasing the replay factor, but that benefit is lesser with higher ρ value.

Intuitively, PER already prioritizes high TD-error transitions, ensuring that informative samples are replayed more frequently. As a result, increasing ρ (i.e., performing more updates per timestep) provides diminishing returns, since the updates are already focused on useful experiences.